# Panduan Teknis 02 - Virtual Environment dan Dependensi

Dokumen ini melanjutkan setup Python dengan memperkenalkan virtual environment dan dependensi. Bagian ini penting karena project AI dan robotika hampir selalu memakai banyak library dengan versi tertentu. Tujuannya adalah membuat satu project dapat dijalankan tanpa merusak project lain.

> **Urutan yang akan dilakukan:** (1) membuat folder project; (2) membuat virtual environment; (3) mengaktifkannya; (4) memeriksa interpreter; (5) memasang library; (6) menyimpan requirement; (7) menjalankan project; dan (8) mendiagnosis konflik jika muncul.

## Langkah 1 - Masuk ke Folder Project

Virtual environment sebaiknya dibuat di dalam folder project yang akan digunakan.

```
project-embodied-ai/
├── main.py
└── requirements.txt
```

Buka terminal di folder tersebut, lalu pastikan lokasi aktif benar.

```
# Linux / macOS
pwd

# Windows
cd
```

## Langkah 2 - Buat Virtual Environment

Gunakan module `venv` yang sudah termasuk dalam Python:

```
python -m venv .venv
```

Perintah ini membuat folder `.venv`. Folder tersebut berisi interpreter dan lokasi instalasi package khusus untuk project ini.

> **Mengapa perlu?** Project A dapat menggunakan versi library yang berbeda dari Project B. Tanpa environment terpisah, upgrade pada satu project dapat membuat project lain berhenti bekerja.

## Langkah 3 - Aktifkan Virtual Environment

| Sistem | Perintah |
|---|---|
| Linux / macOS | `source .venv/bin/activate` |
| Windows PowerShell | `.venv\Scripts\Activate.ps1` |
| Windows Command Prompt | `.venv\ScriptsActivate.bat` |

Setelah aktif, nama environment biasanya muncul di awal prompt, misalnya `(.venv)`.

## Langkah 4 - Pastikan Python dan pip Berasal dari Environment yang Sama

Verifikasi segera sebelum memasang package:

```
python --version
python -m pip --version
```

Pada Linux/macOS, lokasi interpreter dapat diperiksa dengan:

```
which python
```

Pada Windows:

```
where python
```

> **Hasil yang diharapkan.** Path Python mengarah ke folder `.venv`, bukan ke instalasi global.

## Langkah 5 - Pasang Library yang Dibutuhkan

Mulai dari package yang memang diperlukan project. Contoh:

```
python -m pip install numpy matplotlib
```

Periksa hasil instalasi:

```
python -m pip list
python -m pip show numpy
```

Library dapat langsung diuji:

```
python -c "import numpy; print(numpy.__version__)"
```

## Langkah 6 - Pahami Package, Library, dan Framework

| Istilah | Gambaran sederhana | Contoh |
|---|---|---|
| Package | Unit software Python yang dapat dipasang dengan pip. | `numpy`, `pillow` |
| Library | Kumpulan fungsi dan class yang dipanggil dari kode. | NumPy, Matplotlib, OpenCV |
| Framework | Ekosistem yang lebih besar dan biasanya menentukan pola kerja. | PyTorch, Transformers, robosuite |

Dalam praktik, yang paling penting adalah mengetahui nama package, versinya, dan environment tempat package tersebut dipasang.

## Langkah 7 - Simpan Dependensi dalam requirements.txt

Jika project akan dijalankan kembali di komputer lain atau oleh pembaca lain, dependensi sebaiknya direkam.

Buat file `requirements.txt`:

```
numpy==1.26.4
matplotlib>=3.8
transformers==4.49.0
accelerate==0.26.0
```

Instal semua requirement dengan:

```
python -m pip install -r requirements.txt
```

Untuk melihat versi yang sedang terpasang:

```
python -m pip freeze
```

`pip freeze` biasanya lebih panjang karena menampilkan dependensi transitif. Untuk materi edukasi, requirements yang ringkas sering lebih mudah dipelihara selama versi kritis tetap dicatat.

## Langkah 8 - Jalankan Project dari Environment yang Aktif

Setelah dependency terpasang, jalankan script:

```
python main.py
```

Jika script menggunakan PyTorch atau library lain, import terjadi dari environment yang aktif. Karena itu, setiap kali membuka terminal baru, aktifkan environment sebelum menjalankan project.

## Langkah 9 - Keluar dari Environment Setelah Selesai

```
deactivate
```

Perintah ini mengembalikan terminal ke interpreter global. Folder `.venv` tetap ada dan dapat diaktifkan kembali kapan saja.

## Langkah 10 - Kenali Gejala Konflik Dependensi

Konflik terjadi ketika requirement antar-package tidak cocok atau project dijalankan dengan environment yang berbeda dari yang diuji. Gejala umum meliputi:

- `ModuleNotFoundError` padahal package merasa sudah dipasang,
- pesan `requires X, but you have Y`,
- fungsi atau argumen yang hilang setelah upgrade,
- error import setelah satu library baru dipasang,
- error dtype/device pada stack GPU setelah versi PyTorch, Transformers, atau bitsandbytes berubah.

## Langkah 11 - Diagnosis Konflik Secara Berurutan

### 11.1 Catat environment yang benar-benar aktif

```
python --version
python -m pip --version
python -m pip list
```

### 11.2 Baca bagian bawah traceback

Mulai dari baris terakhir untuk melihat error utama, lalu naik beberapa baris untuk mengetahui library dan fungsi yang terlibat.

### 11.3 Periksa requirement resmi project

Cari `requirements.txt`, `pyproject.toml`, README, dokumentasi resmi, atau issue repository. Repository riset sering hanya stabil pada kombinasi versi tertentu.

### 11.4 Ubah satu hal dalam satu waktu

Jika perlu downgrade atau upgrade, lakukan satu perubahan, jalankan kembali, lalu catat hasilnya. Mengubah banyak package sekaligus membuat akar masalah sulit diketahui.

### 11.5 Jalankan pemeriksaan pip

```
python -m pip check
```

Jika ada requirement yang tidak terpenuhi, pip akan menampilkan package yang bermasalah.

## Langkah 12 - Jika Environment Sudah Berantakan, Buat Ulang

Sering kali membuat environment baru lebih cepat dan lebih aman daripada terus memperbaiki environment yang sudah banyak berubah.

Linux/macOS:

```
deactivate
rm -rf .venv
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Windows PowerShell:

```
deactivate
Remove-Item -Recurse -Force .venv
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

> **Hati-hati saat menghapus folder.** Pastikan nama dan lokasi `.venv` benar sebelum menjalankan perintah penghapusan.

## Langkah 13 - Periksa GPU, CUDA, dan PyTorch Secara Terpisah

Jika project memakai GPU, lapisan dependensi tidak berhenti di pip. Kita juga berhadapan dengan driver GPU, build PyTorch, CUDA runtime, dan kadang library quantization.

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("Torch CUDA:", torch.version.cuda)
```

Jika `torch.cuda.is_available()` bernilai `False`, jangan langsung menyalahkan model. Periksa terlebih dahulu apakah runtime menggunakan GPU dan apakah build PyTorch mendukung CUDA.

## Langkah 14 - Gunakan ChatGPT sebagai Asisten Troubleshooting

ChatGPT dapat membantu membaca traceback, membandingkan versi, dan menyusun urutan diagnosis. Agar jawabannya berguna, berikan konteks teknis yang cukup.

```
Saya menjalankan repository <nama-repo>.
OS/runtime: <misalnya Ubuntu 24.04 atau Google Colab T4>
Python: <versi>
PyTorch: <versi>
GPU: <jika relevan>

Perintah/cell yang dijalankan:
<tempel kode>

Traceback lengkap:
<tempel traceback>

Tolong identifikasi akar masalah terlebih dahulu.
Usulkan perubahan satu per satu dan jelaskan alasannya.
```

> **Tetap verifikasi.** Saran AI sebaiknya dibandingkan dengan dokumentasi resmi, README, issue tracker, atau kombinasi versi yang memang diuji oleh pengembang.

## Langkah 15 - Gunakan Pola Environment yang Praktis

Untuk buku ini, satu environment utama dapat menangani hands-on umum. Framework besar yang mempunyai requirement khusus dapat dipisahkan. Contoh:

```
embodied-ai-env/      # Python umum dan hands-on ringan
isaacsim-env/         # simulator dengan requirement khusus
project-x/.venv/      # repository riset tertentu
```

Tujuan akhirnya adalah sederhana: project yang sudah stabil tidak ikut rusak saat kita bereksperimen dengan library atau repository lain.